

Product Requirements Document (PRD)

Cover Page

- **Project Name:** LayoverNow
Course: CISC 593/594
 - **Repository URL:** <https://github.com/pranavapp2024-sys/LayoverNow.git>
 - **Current Branch:** main
 - **Current Commit SHA:** To Be Completed
 - **Current Release Version:** v2.0.0
 - **Document Version:** 1.0
 - **Last Updated:** 2026-07-21
-

Revision History

Version	Date	Git Commit	Description	Author
1.0	2026-07-21	To Be Completed	Initial generation of the PRD based on project README.	AI Assistant

Table of Contents

- 1 [Product Vision](#)
- 2 [Product Scope](#)
- 3 [Software Capabilities](#)
- 4 [Undesirable Events](#)
- 5 [Risk Analysis](#)

6	Risk Prioritization
7	Risk Mitigation
8	Functional Requirements
9	Quality Requirements
10	Performance Requirements
11	Assumptions
12	Constraints
13	External Interfaces
14	Requirements Traceability Matrix
15	Future Versions
16	Open Issues
17	Glossary

1. Product Vision

Problem Statement Standard flight search engines do not easily optimize for multi-day layovers, causing travelers to miss out on free, planned stopovers during flight connections.

Intended Users

- Leisure travelers
- Digital nomads
- Budget-conscious flyers

Stakeholders

- Project developers
- Travelers
- Partner Airlines with official stopover programs (Icelandair, TAP Portugal, Copa, Emirates)

Product Goals

- Surface optimal flight routes whose layover cities are worth stopping in.
- Provide interactive visualizations of candidate flight paths.
- Assist users with layover planning (visa, weather, itinerary) using AI context.

Major Features

- Trip Configuration (Airport autocomplete, date picker, stopover slider)

- Interactive Flight Radar
- Stopover Deals Ranking
- AeroAI Stopover Assistant

Planned Software Versions

- Version 1: Deterministic layover finder (offline)
 - Version 2: AeroAI Advisor and live trip context
-

2. Product Scope

Included functionality

- Offline airport database for origin/destination autocomplete.
- Haversine detour calculation and stopover hub scoring.
- Canvas-based interactive flight radar.
- Stopover Deals results ranked by Best Value, Savings, or Minimum Detour.
- AeroAI Stopover Assistant for visa requirements, currency, weather, and itineraries.
- Booking link generation for partner airlines.
- Graceful offline fallback mechanism.

Excluded functionality

- Hidden-city or skiplagging route calculations.
- Direct booking transactions within the extension.

Future enhancements

- **To Be Completed**
-

3. Software Capabilities

3.1 Level-1 Capabilities

- 1 Configure Trip Parameters

- 2 Calculate Stopover Deals
- 3 Render Flight Radar
- 4 Generate Booking Links
- 5 Manage AI Assistant Context
- 6 Manage Fallback Mechanisms

3.2 Level-2 Capabilities

- 1 Configure Trip Parameters
 - 1.1 Autocomplete Airport Selection
 - 1.2 Select Departure Date
 - 1.3 Adjust Stopover Duration
 - 2 Calculate Stopover Deals
 - 2.1 Calculate Haversine Detour
 - 2.2 Rank Stopover Hubs
 - 3 Render Flight Radar
 - 3.1 Plot Candidate Flight Paths
 - 3.2 Update Active Coordinates
 - 4 Generate Booking Links
 - 4.1 Construct Airline Booking URL
 - 5 Manage AI Assistant Context
 - 5.1 Fetch Visa Requirements
 - 5.2 Fetch Weather Data
 - 5.3 Generate Day-by-Day Itinerary
 - 6 Manage Fallback Mechanisms
 - 6.1 Detect API Failure
 - 6.2 Trigger Deterministic Fallback
-

4. Undesirable Events

UE ID	Level-2 Capability	Undesirable Event
UE-1.1-01	1.1 Autocomplete Airport Selection	Autocomplete fails to find a valid airport code.
UE-1.2-01	1.2 Select Departure Date	User selects a past departure date.

UE ID	Level-2 Capability	Undesirable Event
UE-1.3-01	1.3 Adjust Stopover Duration	User sets stopover duration exceeding airline program limits.
UE-2.1-01	2.1 Calculate Haversine Detour	Haversine calculation fails on antimeridian crossing.
UE-2.2-01	2.2 Rank Stopover Hubs	Hub ranking returns no valid routes.
UE-3.1-01	3.1 Plot Candidate Flight Paths	Flight paths fail to render on the canvas.
UE-3.2-01	3.2 Update Active Coordinates	Active coordinates are stale or incorrect.
UE-4.1-01	4.1 Construct Airline Booking URL	Booking URL parameters are malformed.
UE-5.1-01	5.1 Fetch Visa Requirements	Visa API rate limit exceeded.
UE-5.2-01	5.2 Fetch Weather Data	Weather API returns missing data.
UE-5.3-01	5.3 Generate Day-by-Day Itinerary	AI hallucinates incorrect itinerary details.
UE-6.1-01	6.1 Detect API Failure	API failure detection times out too slowly.
UE-6.2-01	6.2 Trigger Deterministic Fallback	Deterministic fallback fails to activate.

5. Risk Analysis

UE ID	Risk Statement	Likelihood	Impact	Risk Score
UE-1.1-01	If autocomplete fails to find a valid airport code, the user cannot initiate a search.	2	4	8
UE-1.2-01	If the user selects a past departure date, the search will yield no results or error out.	3	3	9
UE-1.3-01	If the stopover duration exceeds airline limits, generated booking links will fail.	3	4	12

UE ID	Risk Statement	Likelihood	Impact	Risk Score
UE-2.1-01	If Haversine calculation fails on antimeridian crossing, incorrect detours will be shown.	2	4	8
UE-2.2-01	If hub ranking returns no valid routes, the extension fails to provide its core value.	2	4	8
UE-3.1-01	If flight paths fail to render, the visual experience is degraded.	2	3	6
UE-3.2-01	If active coordinates are stale, the radar will display incorrect aircraft positions.	3	2	6
UE-4.1-01	If booking URL parameters are malformed, the user cannot complete their booking on the airline site.	2	5	10
UE-5.1-01	If the Visa API rate limit is exceeded, critical trip context is missing.	4	3	12
UE-5.2-01	If the Weather API returns missing data, the user misses environmental context.	3	2	6
UE-5.3-01	If AI hallucinates incorrect itinerary details, the user may rely on false planning information.	3	4	12

UE ID	Risk Statement	Likelihood	Impact	Risk Score
UE-6.1-01	If API failure detection times out too slowly, the UI will freeze and frustrate the user.	3	4	12
UE-6.2-01	If deterministic fallback fails to activate, the entire application becomes unusable during an outage.	2	5	10

6. Risk Prioritization

Priority	UE ID	Risk Score
1	UE-1.3-01	12
2	UE-5.1-01	12
3	UE-5.3-01	12
4	UE-6.1-01	12
5	UE-4.1-01	10
6	UE-6.2-01	10
7	UE-1.2-01	9
8	UE-1.1-01	8
9	UE-2.1-01	8
10	UE-2.2-01	8
11	UE-3.1-01	6
12	UE-3.2-01	6
13	UE-5.2-01	6

7. Risk Mitigation

UE ID	Risk Mitigation	Classification
UE-1.1-01	The Configuration Module shall constrain input to the offline airport database.	Pure Software

UE ID	Risk Mitigation	Classification
UE-1.2-01	The Configuration Module shall disable past dates in the date picker.	Pure Software
UE-1.3-01	The Configuration Module shall cap the stopover slider at 14 days.	Pure Software
UE-2.1-01	The Search Engine shall implement edge-case testing for antimeridian math.	Pure Software
UE-2.2-01	The UI shall display a friendly fallback message if no routes are found.	Pure Software
UE-3.1-01	The Radar Module shall catch canvas exceptions and fail gracefully.	Pure Software
UE-3.2-01	The Radar Module shall interpolate coordinates when active updates are delayed.	Pure Software
UE-4.1-01	The Booking Module shall validate URL parameters against known airline formats.	Pure Software
UE-5.1-01	The AI Context Module shall cache responses to minimize API calls.	Pure Software
UE-5.2-01	The UI shall hide the weather panel if data is unavailable.	Pure Software
UE-5.3-01	The AI Context Module shall prepend a disclaimer about AI-generated content.	Pure Software
UE-6.1-01	The Fallback Module shall enforce a strict 5-second timeout on all API requests.	Pure Software
UE-6.2-01	The application architecture shall use V1 offline capabilities as the default state.	Pure Software

8. Functional Requirements

Requirement ID	Level-2 Capability	Functional Requirement
FR-1.1.1	1.1 Autocomplete Airport Selection	The Configuration Module shall process airport autocomplete queries within 100 milliseconds.

Requirement ID	Level-2 Capability	Functional Requirement
FR-1.2.1	1.2 Select Departure Date	The Configuration Module shall manage departure date selections within standard calendar constraints.
FR-1.3.1	1.3 Adjust Stopover Duration	The Configuration Module shall manage stopover slider bounds within 1 to 14 days.
FR-2.1.1	2.1 Calculate Haversine Detour	The Search Engine shall calculate Haversine detours within 1 second.
FR-2.2.1	2.2 Rank Stopover Hubs	The Search Engine shall perform stopover ranking within offline database limits.
FR-3.1.1	3.1 Plot Candidate Flight Paths	The Radar Module shall plot candidate flight paths within the canvas element.
FR-3.2.1	3.2 Update Active Coordinates	The Radar Module shall manage active coordinate updates within one radar sweep cycle.
FR-4.1.1	4.1 Construct Airline Booking URL	The Booking Module shall construct airline booking URLs within standard HTTP GET constraints.
FR-5.1.1	5.1 Fetch Visa Requirements	The AI Context Module shall fetch visa requirements within a 5-second timeout.
FR-5.2.1	5.2 Fetch Weather Data	The AI Context Module shall fetch weather data within a 5-second timeout.
FR-5.3.1	5.3 Generate Day-by-Day Itinerary	The AeroAI Assistant shall execute layover itinerary generation within API rate limits.
FR-6.1.1	6.1 Detect API Failure	The Fallback Module shall execute API failure detection within 5 seconds of the initial request.
FR-6.2.1	6.2 Trigger Deterministic Fallback	The Fallback Module shall manage the transition to deterministic ranking within 1 second of API failure.

9. Quality Requirements

- **Performance:** The extension popup shall render the initial UI in under 200 milliseconds.
 - **Reliability:** The core offline search engine shall compute correct routing math 100% of the time.
 - **Availability:** The V1 offline layover finder shall be available 100% of the time while the extension is active.
 - **Maintainability:** The codebase shall remain entirely modular vanilla JavaScript without requiring bundlers.
 - **Usability:** The extension shall provide dark mode and glassmorphism styling across all panels.
 - **Security:** The extension shall store API keys exclusively in [chrome.storage.local](#).
 - **Portability:** The extension shall run on any desktop OS supporting Chrome extensions (Manifest V3).
 - **Testability:** The deterministic offline search shall be fully testable without external network dependencies.
 - **AI Explainability:** The AeroAI Assistant shall clearly indicate to the user when content is AI-generated.
-

10. Performance Requirements

- **Maximum response time:** Offline airport autocomplete queries shall complete in < 100ms.
 - **Memory usage:** The active extension process shall consume less than 50MB of RAM.
 - **CPU utilization:** The flight radar canvas rendering shall consume less than 10% CPU.
 - **AI inference latency:** External API calls to the LLM must complete within 5 seconds before a fallback is triggered.
-

11. Assumptions

- Users have Google Chrome installed.

- Users can provide valid API keys for the AeroAI Assistant and weather integration (V2).
 - The provided offline airport database is sufficiently accurate and up-to-date.
 - Stopover policies for supported airlines remain substantially similar to current implementations.
-

12. Constraints

- **Programming language:** Vanilla JavaScript, HTML, CSS.
 - **Framework:** No external frameworks or bundlers.
 - **Extension standard:** Manifest V3 API.
 - **Storage:** Data persistence restricted to [`chrome.storage.local`](#).
 - **External APIs:** Relies on third-party LLM and weather APIs for V2 functionality.
-

13. External Interfaces

User Interfaces

- Chrome Extension Popup (HTML/CSS/JS)
- Interactive Flight Radar Canvas

Hardware Interfaces

- **To Be Completed** (Typically None for a browser extension beyond standard mouse/keyboard input).

Software Interfaces

- Chrome Extension API (Manifest V3, [`chrome.storage.local`](#))

Communication Interfaces

- HTTP/HTTPS for external API requests (AeroAI, Weather) and airline booking redirects.

External Services

- AI LLM API (User-provided via [aiApiKey](#))
- Weather API (User-provided via [weatherApiKey](#))

14. Requirements Traceability Matrix

Requirement ID	Level-2 Capability	Requirement Description
FR-1.1.1	1.1 Autocomplete Airport Selection	Autocomplete within 100ms.
FR-1.2.1	1.2 Select Departure Date	Date management within calendar bounds.
FR-1.3.1	1.3 Adjust Stopover Duration	Slider constraints (1-14 days).
FR-2.1.1	2.1 Calculate Haversine Detour	Haversine calculation within 1 second.
FR-2.2.1	2.2 Rank Stopover Hubs	Rank execution within database limits.
FR-3.1.1	3.1 Plot Candidate Flight Paths	Plot paths on canvas.
FR-3.2.1	3.2 Update Active Coordinates	Coordinate updates during radar sweeps.
FR-4.1.1	4.1 Construct Airline Booking URL	URL construction using GET parameters.
FR-5.1.1	5.1 Fetch Visa Requirements	Visa fetch within 5s.
FR-5.2.1	5.2 Fetch Weather Data	Weather fetch within 5s.
FR-5.3.1	5.3 Generate Day-by-Day Itinerary	Itinerary generation within rate limits.
FR-6.1.1	6.1 Detect API Failure	Detection within 5s of API call.
FR-6.2.1	6.2 Trigger Deterministic Fallback	Transition to fallback within 1s.

15. Future Versions

- **Version 1:** Deterministic layover finder. Airport search, Haversine detour engine, static stopover-program scoring, flight radar, booking link generation. Fully offline.
- **Version 2:** Adds the AeroAI Advisor (AI-assisted layover evaluation) and live trip context (visa, currency, weather). Falls back to V1 ranking when a service is unavailable.
- **Version 3: To Be Completed**
- **Future enhancements: To Be Completed**

16. Open Issues

- **To Be Completed:** Which specific LLM API and Weather API providers are targeted by default?
 - **To Be Completed:** Are there limits on the maximum distance evaluated by the Haversine detour engine?
-

17. Glossary

- **Manifest V3:** The latest extension platform specification for Google Chrome.
- **Haversine Formula:** Mathematical formula used to calculate the great-circle distance between two points on a sphere given their longitudes and latitudes.
- **AeroAI Assistant:** The V2 intelligent assistant module that provides contextual information (visa, weather, itinerary) for a layover.
- **Deterministic Ranking:** The V1 offline logic that ranks layovers based on fixed math (detour distance, savings) rather than AI inference.
- **Hidden-city / Skiplagging:** A ticketing strategy where a passenger disembarks at a connection rather than the final destination (explicitly excluded from this project).